Story Coverage Mechanism — Implementation Brief

How code gets checked against a user story, with and without one. Extends the earlier two-track design (grounding vs. coverage) with implementation-level detail for Track B.

Shared data model (both branches write here)

One table drives judgment, rollup, and reporting for both branches — this is the key integration point, not two parallel systems.

Table	Key columns	Purpose
`story_requirements`	`text`, `type` (positive/negative), `source` (document / derived / human_confirmed), `status` (supported / contradicted / unknown / needs_human_input), `journey_id` (nullable)	One row per atomic, checkable claim, regardless of origin
`requirement_citations`	`requirement_id`, `node_id`, `rationale`	Grounding evidence — same citation pattern already specified for generated test cases
`journeys`	`name`, `entry_point_ids`, `node_ids`	Branch B only — a traced candidate journey; a document-sourced story doesn't need one

Branch A — a user story exists

1. **Decompose into atomic requirements.** Structured input (bulleted acceptance criteria, Given/When/Then) parses deterministically, no LLM. Freeform prose gets one LLM call *per story* (not per module) to extract atomic claims, tagged positive/negative — this call only splits the text, it never judges anything.
2. **Retrieve candidates before judging.** Embed each requirement and search against an index built from text TestAtlas already generates — module friendly names/descriptions, extracted UI strings, route paths — returning ~8–10 candidate nodes. This is what makes English-vs-JS/TS comparison tractable: never compare against raw source directly, and reuse an asset (plain-English module descriptions) that already exists for an unrelated reason. Negative requirements ("X can't happen") retrieve the positive capability they qualify and leave the guard-exists-or-not call to the next step, with a lower expected confidence by design.
3. **Judge with one LLM call per requirement.** Given the requirement and its candidate shortlist (real source, not descriptions): cite the specific function, quote the supporting/contradicting line, answer supported / contradicted / unknown — never a confidence number. Cost is bounded: an 8-criterion story is 8 calls, not one per module.
4. **Roll up.** `supported / total` → story %. For each supported requirement, check whether any existing or generated test case cites the same node → tested yes/no. Produces the implemented × tested 2x2 per story.

Branch B — no story exists

1. **Detect entry points — deterministic, per-framework, not an LLM call.** Route decorators (FastAPI/Express-style), file-path convention (Next.js-style page routing), or command registration (CLI tools) — pattern-matched over the AST data TestAtlas already parses for the graph.
2. **Trace the call chain from each entry point.** Reuses the existing call graph; bounded traversal collects "everything that runs for this request." Nodes touched by many unrelated entry points (shared loggers, generic validators) are deprioritized by a call-frequency threshold.

CONFIDENTIAL

3. **Cluster entry points into journeys.** Combined similarity score: shared downstream nodes (Jaccard), shared route prefix, sequential/redirect linkage — standard hierarchical clustering, the same category of technique already used for module detection, just on a different input. An LLM is used only at the end, only to name the cluster ("Checkout flow") — the grouping decision itself stays deterministic graph math.
4. **Derive requirements from within each journey.** The same pattern-detection approach as step 1, now looking for validation checks, permission checks, and state transitions inside the journey's subgraph. Each match becomes a `story_requirements` row with `source = derived` — written into the *same table* Branch A uses, so judgment and rollup above run unmodified.
5. **Abstain and ask where code can't resolve intent.** A detected pattern with no way to tell if it's correct ("cancellation after dispatch — bug or feature?") gets `status = needs_human_input` instead of a guess, surfaced as a short question in the existing review UI. An answer is written back with `source = human_confirmed` — from that point on it behaves exactly like a Branch A requirement on every future run. That's the actual mechanism behind "answers compound over time": a `source` field transition in one shared table.

Reporting

One dashboard over `story_requirements`, regardless of branch: requirement, source, status, implemented %, tested %, review status. The `source` column is what lets a reviewer tell a human-confirmed fact from a code-derived guess still waiting on confirmation.

Known limitations

- Retrieval quality bounds everything downstream — if step A2 misses the right code, A3's judgment is wrong regardless of prompt quality.
- Negative requirements structurally land in "unknown" more often — retrieval finds what exists, not the absence of a violation.
- Entry-point/pattern detection is real, ongoing per-framework engineering, not a one-time cost.

CONFIDENTIAL

- Clustering in B3 is heuristic and will sometimes merge or split journeys wrong — shown as uncertain, not asserted as settled.

Build order

Branch A first (reuses existing assets almost entirely — only new pieces are the embedding index and the per-requirement judgment call). Build Branch B's data model (`source=derived`, `journey_id`) compatible with Branch A from day one, even before any detector exists. Add Branch B's detectors last, one framework at a time, starting with whatever real customers actually run.

CONFIDENTIAL